

Thực hành – Tuần 2A

Cú pháp & Ngữ nghĩa



Viện Trí tuệ Nhân tạo, Trường Đại học Công nghệ, ĐHQGHN

Ngày 9 tháng 3 năm 2026

Mục lục

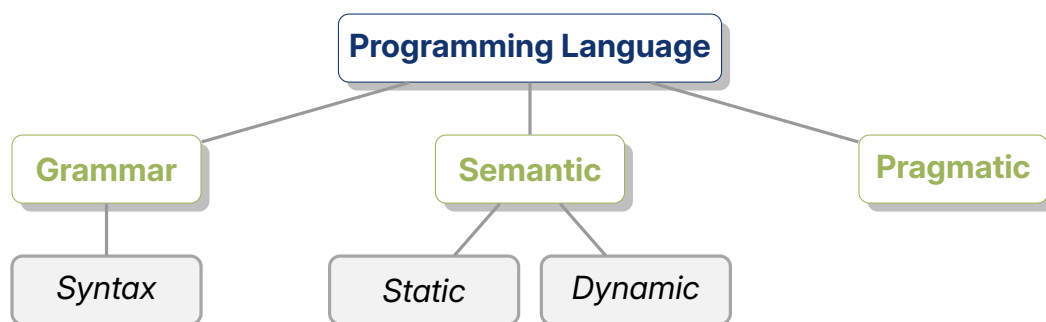
I	Nội dung thực hành	2
I.1	Kiến thức	2
I.1.1	Mô tả một ngôn ngữ lập trình	2
I.1.2	Quá trình biên dịch	7
I.2	Mục tiêu	9
II	Bài tập thực hành	10
III	Bài tập về nhà	17

I. Nội dung thực hành

I.1 Kiến thức

Trong bài thực hành tuần trước, chúng ta đã làm quen với khái niệm *Máy trừu tượng* (*Abstract Machine*) của một ngôn ngữ lập trình và hai phương thức thực thi chính: *Thông dịch* (*Interpretation*) và *Biên dịch* (*Compilation*). Tuy nhiên, để hiểu sâu hơn cách một chương trình được hiểu và thực thi bởi máy tính, chúng ta cần xem xét cách *ngôn ngữ lập trình được mô tả và quy trình biên dịch mã nguồn*.

I.1.1 Mô tả một ngôn ngữ lập trình



Hình 1: Các khía cạnh chính của một ngôn ngữ lập trình

Một ngôn ngữ lập trình không chỉ đơn thuần là tập hợp các ký tự mà lập trình viên viết ra; nó được mô tả thông qua nhiều tầng khác nhau, bao gồm *Ngữ pháp* (*Grammar*), *Ngữ nghĩa* (*Semantics*), và *Ngữ dụng* (*Pragmatics*). Các tầng này giúp trả lời ba câu hỏi cơ bản:

- **Ngữ pháp:** “Một câu lệnh nào được coi là hợp lệ?”
- **Ngữ nghĩa:** “Một câu lệnh hợp lệ có ý nghĩa gì?”
- **Ngữ dụng:** “Sử dụng câu lệnh đó trong thực tế như thế nào cho hợp lý?”

Để minh họa các khái niệm này, chúng ta bắt đầu với một chương trình đơn giản quen thuộc trong C++:

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     cout << "Hello World";
7     return 0;
8 }
```

Ngữ pháp (Grammar) và Cú pháp (Syntax) Khi nhìn vào chương trình trên, máy tính không hiểu trực tiếp các dòng mã như con người. Thay vào đó, mã nguồn được xem như một chuỗi ký tự. Trình biên dịch trước tiên sẽ nhóm các ký tự này thành các *token* (thành phần từ vựng), ví dụ:

```
int    main    (    )    {    std    ::    cout    <<    "Hello World"    ;    }
```

Trong giai đoạn *phân tích từ vựng (Lexical Analysis)*, mỗi token được biểu diễn dưới dạng một bộ gồm *loại token (token type)*, *giá trị từ vựng (lexeme)* và *định danh nội bộ (token value)* mà trình biên dịch sử dụng. Ví dụ:

Token Type	Lexeme	Token Value (Clang)
Keyword	int	kw_int
Identifier	main	identifier
Left Parenthesis	(	l_paren
Right Parenthesis	)	r_paren
Left Brace	{	l_brace
Identifier	std	identifier
Scope Operator	::	coloncolon
Identifier	cout	identifier
Stream Operator	<<	lessless
String Literal	"Hello World"	string_literal
Semicolon	;	semi
Right Brace	}	r_brace

Kết quả của bước này là một danh sách token được chuyển tiếp cho giai đoạn tiếp theo của trình biên dịch.

Sau khi có danh sách token, trình biên dịch sẽ kiểm tra xem chúng có tuân theo *cú pháp* của ngôn ngữ hay không. Cú pháp được mô tả bằng các quy tắc văn phạm; cụ thể hơn là *văn phạm phi ngữ cảnh (Context-Free Grammar)*, ví dụ một cách đơn giản:

```
1 <program> ::= <function>
2
3 <function> ::= int main ( ) { <statement> }
4
5 <statement> ::= std :: cout << <string> ;
```

Nếu chương trình vi phạm các quy tắc này, trình biên dịch sẽ báo *lỗi cú pháp (syntax error)*. Ví dụ:

```
1 int main( {
```

Trong trường hợp này, dấu ngoặc không hợp lệ nên chương trình không thể được phân tích cú pháp.

Ngữ nghĩa (Semantics) Sau khi chương trình vượt qua giai đoạn kiểm tra cú pháp, trình biên dịch cần xác định *ý nghĩa* của các cấu trúc trong chương trình. Đây là nhiệm vụ của tầng *ngữ nghĩa (semantics)*. Nói cách khác, ngữ nghĩa trả lời câu hỏi: “Một chương trình đúng cú pháp sẽ thực sự *làm gì?*”.

Trong thực tế, ngữ nghĩa của một ngôn ngữ lập trình thường được chia thành hai loại: *ngữ nghĩa tĩnh (static semantics)* và *ngữ nghĩa động (dynamic semantics)*.

Ngữ nghĩa tĩnh (Static Semantics) Như đã đề cập ở phần trước, cú pháp của ngôn ngữ lập trình thường được mô tả bằng *văn phạm phi ngữ cảnh*. Tuy nhiên, không phải mọi quy tắc hợp lệ của một ngôn ngữ đều có thể được mô tả bằng loại văn phạm này.

Trong thực tế, có thể thấy sự tồn tại của nhiều *ràng buộc phụ thuộc vào ngữ cảnh (contextual constraints)* ở rất nhiều ngôn ngữ lập trình khác nhau, ví dụ như:

- Một định danh phải được *khai báo* trước khi được sử dụng.
- Kiểu dữ liệu của hai toán hạng phải *tương thích* với toán tử.
- Một hàm được gọi phải tồn tại và có *chữ ký* phù hợp.

Những quy tắc này không thể được biểu diễn trực tiếp bằng văn phạm phi ngữ cảnh. Vì vậy, chúng được tách ra thành một lớp kiểm tra riêng gọi là *ngữ nghĩa tĩnh*. Giai đoạn này thường được thực hiện trong bước *phân tích ngữ nghĩa (Semantic Analysis)* của trình biên dịch.

Ví dụ:

```
1 int main() {  
2     std::cout << x;  
3 }
```

Chương trình trên có cú pháp hoàn toàn hợp lệ. Tuy nhiên, biến *x* chưa được khai báo trước đó. Đây là một lỗi *ngữ nghĩa tĩnh (static semantic error)*, vì ràng buộc “định danh phải được khai báo trước khi sử dụng” đã bị vi phạm. Trình biên dịch có thể phát hiện lỗi này trực tiếp từ mã nguồn mà không cần thực thi chương trình.

Ngữ nghĩa động (Dynamic Semantics) Trong khi đó, *ngữ nghĩa động* mô tả hành vi của chương trình trong quá trình *thực thi*. Nó giải thích cách mà trạng thái của máy trừu tượng thay đổi khi các lệnh được thực hiện. Nói cách khác, ngữ nghĩa động trả lời câu hỏi: “Điều gì xảy ra khi chương trình chạy?”.

Khác với ngữ nghĩa tĩnh, các vấn đề liên quan đến ngữ nghĩa động thường chỉ có thể được phát hiện *trong quá trình chạy chương trình*. Điều này là do việc xác định lỗi đòi hỏi phải biết trạng thái cụ thể của chương trình tại thời điểm thực thi.

Ví dụ, câu lệnh:

```
1 std::cout << "Hello World";
```

về mặt ngữ nghĩa động có thể được hiểu như một thao tác ghi chuỗi ký tự ra thiết bị xuất chuẩn của hệ thống.

Trong một số trường hợp khác, lỗi chỉ xuất hiện khi chương trình thực sự chạy, chẳng hạn như truy cập vào vùng nhớ không hợp lệ hoặc chia cho số 0. Những lỗi này không thể luôn luôn được phát hiện ở giai đoạn kiểm tra mã nguồn, mà cần được *suy luận* (*reasoning*) dựa trên trạng thái thực thi của chương trình.

Ví dụ:

```
1 #include <iostream>
2
3 int main() {
4     int a = 10;
5     int b = 0;
6
7     int c = a / b;
8     std::cout << c << std::endl;
9 }
```

Chương trình trên hoàn toàn đúng về cú pháp và cũng không vi phạm các ràng buộc ngữ nghĩa tĩnh. Tuy nhiên, tại thời điểm thực thi, biến `b` có giá trị bằng 0, dẫn tới phép chia không hợp lệ. Lỗi này chỉ có thể được phát hiện khi chương trình chạy và giá trị thực tế của các biến đã được xác định. Do đó, đây là một ví dụ điển hình của vấn đề thuộc *ngữ nghĩa động*.

Tóm lại, có thể phân biệt hai loại ngữ nghĩa như sau:

- **Ngữ nghĩa tĩnh:** các ràng buộc có thể kiểm tra trực tiếp từ mã nguồn trước khi chương trình chạy.
- **Ngữ nghĩa động:** hành vi và các lỗi chỉ xuất hiện trong quá trình thực thi chương trình.

Ngữ dụng (Pragmatics) Ngữ dụng không liên quan trực tiếp tới việc chương trình có hợp lệ hay không, mà tập trung vào mục đích và cách sử dụng các cấu trúc của ngôn ngữ một cách hiệu quả và dễ hiểu. Khác với cú pháp và ngữ nghĩa – vốn được định nghĩa chính xác trong đặc tả của ngôn ngữ – ngữ dụng không phải là một tập quy tắc cố định. Thay vào đó, nó phát triển dần theo kinh nghiệm sử dụng của cộng đồng lập trình viên và thực tiễn phát triển phần mềm.

Ví dụ, hai đoạn chương trình sau đều đúng:

```
1 std::cout << "Hello";
2 std::cout << "World";
```

và

```
1 std::cout << "Hello World";
```

Tuy nhiên, cách viết thứ hai thường được xem là hợp lý hơn vì dễ đọc và giảm số lần thao tác xuất dữ liệu. Những lựa chọn như vậy thuộc về *ngữ dụng* của ngôn ngữ.

Một ví dụ kinh điển khác trong ngữ dụng là việc sử dụng câu lệnh goto. Trong nhiều ngôn ngữ lập trình, goto cho phép chương trình nhảy trực tiếp tới một vị trí khác trong mã nguồn. Tuy nhiên, trong thực hành lập trình hiện đại, goto thường được khuyến cáo nên tránh sử dụng vì nó làm phá vỡ cấu trúc điều khiển của chương trình và khiến luồng thực thi trở nên khó theo dõi. Điều này có thể làm giảm khả năng đọc hiểu và bảo trì mã nguồn, đặc biệt trong các hệ thống phần mềm lớn.

Ví dụ, một vòng lặp for trong C++:

```
1 for (int i = 0; i < 3; i++) {  
2     std::cout << i << std::endl;  
3 }
```

khi được biên dịch xuống mức hợp ngữ, về bản chất vẫn được thực hiện thông qua các lệnh nhảy điều kiện, tương đương với một dạng goto ở mức thấp:

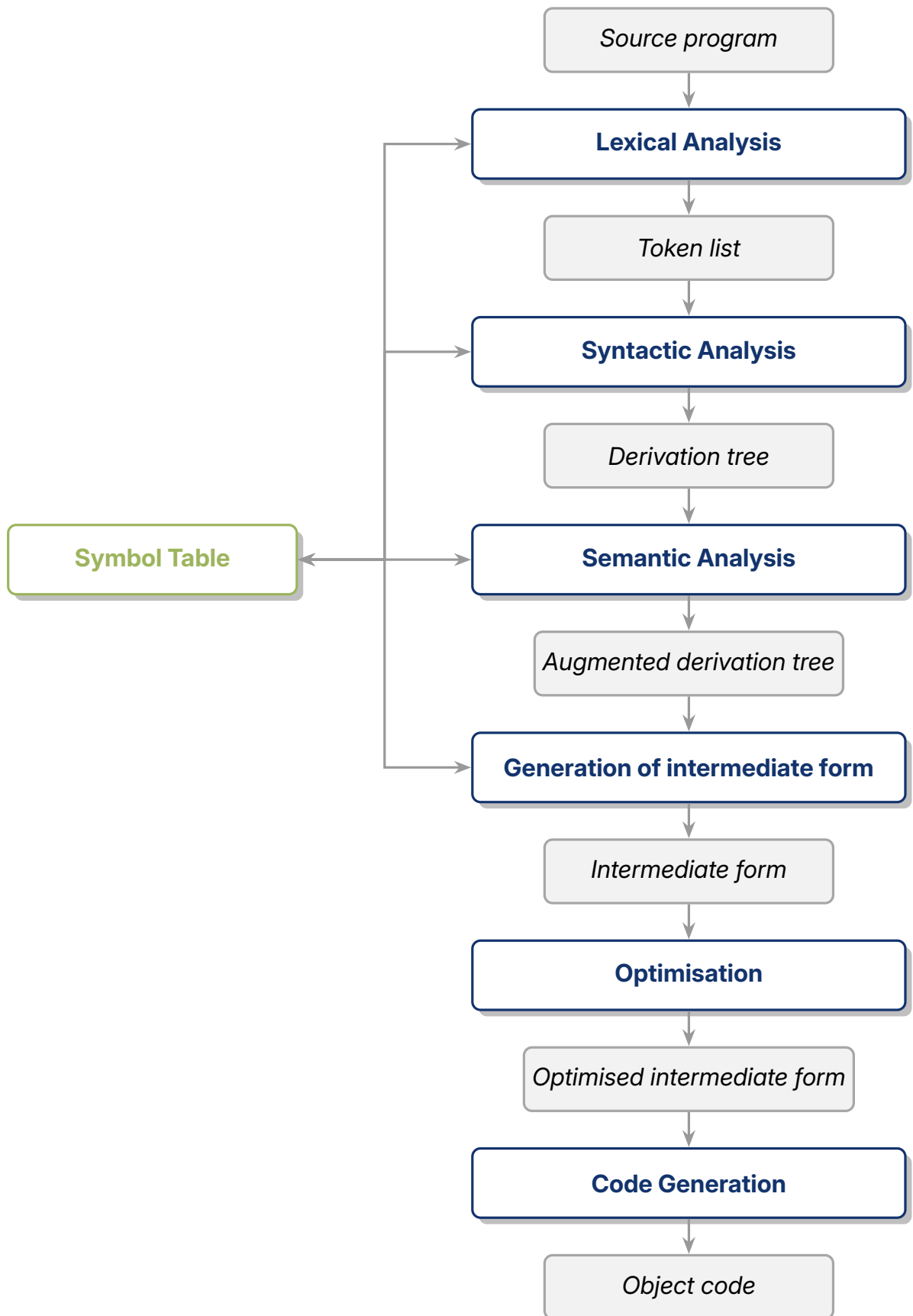
```
1 loop_start:  
2     if (i >= 3) goto loop_end  
3     print(i)  
4     i = i + 1  
5     goto loop_start  
6 loop_end:
```

Điều này cho thấy rằng cơ chế nhảy (*jump*) vẫn là một thành phần cơ bản của các ngôn ngữ bậc thấp và kiến trúc phần cứng. Tuy nhiên, các ngôn ngữ lập trình bậc cao như C++ cung cấp các cấu trúc điều khiển có tổ chức hơn (structured control flow) như if, while, hoặc for để giúp chương trình trở nên rõ ràng và dễ bảo trì hơn.

Vì vậy, việc “tránh dùng goto” không phải là một quy tắc cú pháp bắt buộc của ngôn ngữ, mà chỉ là một khuyến nghị thuộc về *ngữ dụng*. Nói cách khác, ngữ dụng không cấm một cấu trúc, mà đưa ra những hướng dẫn về cách sử dụng cấu trúc đó sao cho phù hợp trong thực tế.

Dễ thấy nhất là trong các môi trường phát triển phần mềm chuyên nghiệp, những nguyên tắc này thường được đúc kết và cụ thể hóa thành các *quy ước lập trình* (*coding conventions*) hoặc *phong cách lập trình* (*coding style*). Nhiều công ty công nghệ và dự án phần mềm lớn đều xây dựng các bộ quy tắc riêng cho mã nguồn của họ, nhằm đảm bảo rằng các lập trình viên có thể viết mã theo một cách nhất quán, dễ đọc và dễ bảo trì. Do đó, có thể xem ngữ dụng như điểm giao thoa giữa việc nghiên cứu ngôn ngữ lập trình và các phương pháp thực hành trong *kỹ nghệ phần mềm* (*software engineering*).

I.1.2 Quá trình biên dịch



Hình 2: Các giai đoạn của một trình biên dịch

Từ các khái niệm trên, có thể thấy rằng việc chuyển đổi một chương trình từ mã nguồn sang dạng thực thi không phải là một bước duy nhất, mà là một chuỗi các giai đoạn xử lý liên tiếp. Trong thực tế, một trình biên dịch hiện đại thường được tổ chức thành nhiều tầng xử lý.

Quá trình này có thể được mô tả khái quát như trong Hình 1.1.2. Mỗi giai đoạn trong sơ đồ đảm nhận một nhiệm vụ chuyên biệt nhằm chuyển đổi dần mã nguồn từ ngôn ngữ bậc cao sang ngôn ngữ máy:

- **Lexical Analysis:** Trình biên dịch "đọc" mã nguồn và cắt nhỏ chúng thành các **Token**. Đây là đơn vị nhỏ nhất mà máy tính có thể định nghĩa (như từ khóa `int`, toán tử `<<`).
- **Syntactic Analysis:** Các Token được sắp xếp vào một cây cú pháp (**Derivation Tree**). Bước này kiểm tra xem các Token có đặt đúng trật tự quy định (như dấu ngoặc có đóng mở đủ hay không).
- **Semantic Analysis:** Trình biên dịch kiểm tra ý nghĩa của câu lệnh (như biến đã được khai báo chưa, kiểu dữ liệu có khớp không) và bổ sung thông tin vào cây (**Augmented Tree**).
- **Intermediate Form Generation:** Cây cú pháp được làm phẳng thành mã trung gian (**GIMPLE/IR**). Dạng mã này không phụ thuộc vào phần cứng cụ thể, giúp trình biên dịch dễ dàng phân tích logic.
- **Optimisation:** Đây là giai đoạn "làm sạch" mã trung gian. Trình biên dịch loại bỏ mã thừa (*Deadcode*), gộp các phép tính lặp lại hoặc tính toán trước các hằng số để chương trình chạy nhanh hơn.
- **Code Generation:** Cuối cùng, mã trung gian đã tối ưu được chuyển đổi thành mã hợp ngữ (**Assembly**) hoặc mã máy (**Object code**) tương thích với kiến trúc CPU của máy tính.

Xuyên suốt các giai đoạn này, một **Symbol Table** (Bảng ký hiệu) được sử dụng để lưu trữ và tra cứu thông tin về các biến, hàm và phạm vi của chúng, đảm bảo tính nhất quán cho toàn bộ chương trình.

Tóm lại, khi chúng ta viết một chương trình C++, trình biên dịch không chỉ đơn thuần chuyển đổi mã nguồn sang mã máy. Nó phải trải qua nhiều bước để đảm bảo rằng chương trình:

- tuân theo **cú pháp** của ngôn ngữ,
- có **ngữ nghĩa** hợp lệ,
- và cuối cùng có thể được chuyển đổi thành các lệnh thực thi trên phần cứng.

I.2 Mục tiêu

Trong bài tập tuần này, các bạn sẽ quan sát cụ thể từng bước của quá trình biên dịch mã nguồn thông qua việc tương tác trực tiếp với trình biên dịch g++ và clang. Buổi học giúp các bạn hiểu rõ cơ chế vận hành nội tại của Compiler thay vì chỉ coi nó là một "hộp đen". Cụ thể, các bạn cần đạt được các mục tiêu sau:

- **Phân tích từ vựng (Lexical Analysis):** Biết cách trích xuất danh sách **Tokens** từ mã nguồn, nhận diện các đơn vị cơ bản như từ khóa, định danh và toán tử.
- **Phân tích cú pháp và ngữ nghĩa (Syntactic & Semantic Analysis):** Tiếp cận cấu trúc cây **AST (Abstract Syntax Tree)** để hiểu cách trình biên dịch kiểm tra tính đúng đắn của câu lệnh và quản lý **Symbol Table** (Bảng ký hiệu).
- **Mã trung gian (Intermediate Representation):** Làm quen với dạng mã ba địa chỉ (**GIMPLE**), hiểu cách các biểu thức phức tạp được phân rã thành các bước tính toán đơn giản, tạo tiền đề cho việc tối ưu hóa.
- **Tối ưu hóa mã nguồn (Optimization):** Quan sát và phân tích các kỹ thuật tối ưu hóa kinh điển dựa trên lý thuyết:
 - *Deadcode removal*: Loại bỏ mã thừa không ảnh hưởng đến kết quả.
 - *Subexpression factorisation*: Tái sử dụng kết quả của các biểu thức lặp lại.
 - *In-line expansion*: Triệt tiêu chi phí gọi hàm bằng cách chèn trực tiếp nội dung hàm vào nơi sử dụng.
- **Sinh mã (Code Generation):** Đọc hiểu mã **Assembly** cơ bản để thấy sự tương quan giữa các lệnh điều khiển cấp cao và các thao tác trực tiếp trên thanh ghi (Registers) cũng như bộ nhớ (Stack) của CPU.

Thông qua việc so sánh sự khác biệt của mã nguồn ở từng giai đoạn (đặc biệt là trước và sau khi tối ưu hóa), các bạn sẽ nắm vững mối liên hệ giữa hiệu năng chương trình và cách thức trình biên dịch xử lý mã.

II. Bài tập thực hành

Bài tập 1: Quan sát danh sách Token của chương trình

Mô tả:

Tạo một chương trình C++ đơn giản và quan sát danh sách các token mà trình phân tích từ vựng (Lexical Analysis) sinh ra.

Yêu cầu kỹ thuật:

- Sử dụng clang để xuất danh sách token.
- Chương trình cần có ít nhất:
 - một hàm main
 - một biến kiểu int
 - một câu lệnh return
- Thực hiện lệnh:

```
1 clang -Xclang -dump-tokens source.cpp
```

Input:

Không có dữ liệu đầu vào.

Output:

Danh sách các token của chương trình, ví dụ:

```
int 'int'
identifier 'main'
l_paren '('
r_paren ')'
l_brace '{'
...
```

Câu hỏi thảo luận:

- Một token gồm những thành phần nào?
- Tại sao khoảng trắng không xuất hiện trong danh sách token?

Lưu ý:

Để kết quả không bị chứa nhiều thông tin nhiễu, thêm đoạn sau thêm vào ... source.cpp khi chạy:

- Với Windows (PowerShell):

```
1 ... source.cpp 2>\&1 | findstr "source.cpp" > tokens_clean.txt
```

- Với Linux/macOS:

```
1 ... source.cpp 2>\&1 | grep "source.cpp" > tokens_clean.txt
```

Sau đó truy cập file `tokens_clean.txt` để xem kết quả.

Cài đặt clang theo hướng dẫn như sau:

- Với Windows (MSYS2 UCRT64):

```
1 pacman -S mingw-w64-ucrt-x86_64-clang
```

- Với Linux (Ubuntu/Debian):

```
1 sudo apt update && sudo apt install clang
```

- Với MacOS đã có sẵn clang thông qua cài đặt bằng lệnh `xcode-select --install`.

Bài tập 2: Quan sát cây cú pháp (Abstract Syntax Tree)

Mô tả:

Sau khi mã nguồn được phân tích từ vựng, trình biên dịch xây dựng một cây cú pháp trừu tượng (AST). Bài tập này yêu cầu quan sát cấu trúc AST được sinh ra bởi g++.

Yêu cầu kỹ thuật:

- Viết chương trình C++ khai báo 2 biến `int a = 5` và `int b = 10`, tính tổng `int c = a + b`.
- Biên dịch với lệnh:

```
1 g++ -fdump-tree-original source.cpp
```

- Mở file `source.cpp.*.original` được sinh ra.

Input:

Hai số nguyên được khai báo trong chương trình.

Output:

File chứa biểu diễn cây cú pháp của chương trình.

```
{
  int a = 5;
  int b = 10;
  int c = a + b;
  <<cleanup_point int a = 5;>>;
  ...
  return <retval> = 0;
}
```

Câu hỏi thảo luận:

- Tại sao trong file kết quả AST lại xuất hiện ký hiệu `<retval>` và các cụm `cleanup_point`? Chúng đóng vai trò gì trong phân tích ngữ nghĩa?
- Cấu trúc cây trong file `.original` có khác biệt gì so với mã nguồn ban đầu (về thứ tự thực thi hoặc cách biểu diễn phép gán)?

Lưu ý:

Trong mã nguồn `source.cpp`, không nên sử dụng `iostream` để giảm tính phức tạp của cây AST.

Bài tập 3: Quan sát mã trung gian (Intermediate Representation)

Mô tả:

Trình biên dịch chuyển đổi cây cú pháp thành một dạng mã trung gian để dễ tối ưu hóa. Trong GCC, dạng này thường là GIMPLE.

Yêu cầu kỹ thuật:

- Từ mã nguồn của Bài tập 2, viết thêm dòng lệnh tính biểu thức sau trước khi `return 0;`:

```
1 int x = a + b * c;
```

- Biên dịch bằng lệnh:

```
1 g++ -fdump-tree-gimple source.cpp
```

- Mở file `source.cpp.*.gimple` được sinh ra.

Input:

Không có dữ liệu đầu vào.

Output:

Mã trung gian được đơn giản hóa về dạng các phép tính cơ bản:

```
c = a + b;  
D.2345 = b * c;           // Một địa chỉ mới được tạo ra  
x = a + D.2345;
```

Câu hỏi thảo luận:

- Tại sao các biểu thức phức tạp được tách thành các biến tạm (ví dụ `D.2345`)?
- Dạng mã này giúp ích gì cho việc tính toán trên các thanh ghi của CPU?

Bài tập 4: Quan sát quá trình tối ưu hóa (Optimisation)

Mô tả:

Trong bài tập này, chúng ta sẽ quan sát cách trình biên dịch thực hiện tối ưu hóa trên mã nguồn có dữ liệu động (nhập từ cin). Các kỹ thuật cần quan sát bao gồm:

- **Deadcode removal:** Loại bỏ biến hoặc phép tính không đóng góp vào kết quả cuối cùng.
- **Subexpression factorisation:** Phát hiện và tính toán các biểu thức lặp lại (như $a + b$) duy nhất một lần.
- **In-line expansion:** Thay thế lời gọi hàm bằng nội dung thực thi của chính hàm đó để tăng tốc độ.

Yêu cầu kỹ thuật:

- Sử dụng mã nguồn chứa hàm `inline int square(int n)` và các phép tính:

```
1  #include <iostream>
2
3  inline int square(int n) {
4      return n * n;
5  }
6
7  int main() {
8      // Avoid compiler calc value
9      int a, b; std::cin >> a >> b;
10
11     // Deadcode removal
12     int c = 10;
13
14     // Subexpression factorisation
15     int x = (a + b) * (a + b);
16     int y = (a + b) + c;
17
18     // In-line expansion
19     int z = square(a);
20
21     return x + y + z;
22 }
```

- Biên dịch và xuất file tối ưu hóa mức độ 2 (-O2) cuối cùng:

```
1  g++ -O2 -fdump-tree-optimized source.cpp
```

- Mở file `source.cpp.*.optimized` để phân tích.

Input:

Không yêu cầu dữ liệu đầu vào.

Output (Dấu hiệu nhận biết trong file `.optimized`):

```
_3 = a + b;           // Subexpression factorisation
...
x_9 = _3 * _3;        // Tái sử dụng kết quả _3
...
y_10 = _3 + 10;       // Deadcode removal: c biến mất
...
_11 = a * a;          // In-line expansion: square biến mất
...
```

1. **Subexpression factorisation:** Biểu thức $(a + b)$ chỉ được tính một lần và lưu vào biến tạm `_3`, sau đó `_3` được dùng để tính cả `x` và `y`.
2. **Deadcode removal:** Biến `c` và phép gán `c = 10` đã bị loại bỏ hoàn toàn, trình biên dịch thay trực tiếp hằng số `10` vào phép tính `y_10`.
3. **In-line expansion:** Lời gọi hàm `square(a)` không còn tồn tại; thay vào đó, nội dung hàm được "nội tại hóa" thành phép nhân `a * a` (biến tạm `_11`).

Câu hỏi thảo luận:

- **Deadcode removal:** Tại sao trình biên dịch lại thay `c` bằng số `10` trực tiếp vào biểu thức của `y` thay vì giữ lại biến `c`?
- **Subexpression factorisation:** Hãy chỉ ra biến tạm (ví dụ `_1` hoặc `D.xxxx`) trong file dump đại diện cho $(a + b)$. Nó được tái sử dụng bao nhiêu lần?
- **In-line expansion:** Tại sao việc "dán" code của hàm `square` trực tiếp vào `main` lại giúp chương trình chạy nhanh hơn việc gọi hàm thông thường?

Bài tập 5: Quan sát mã Assembly (Code Generation)

Mô tả:

Giai đoạn cuối cùng của trình biên dịch là chuyển đổi mã trung gian đã tối ưu hóa thành mã hợp ngữ (Assembly) dành riêng cho một kiến trúc CPU cụ thể (như x86_64).

Yêu cầu kỹ thuật:

- Sử dụng chương trình tính tổng ở Bài tập 2 (không dùng cịn để mã Assembly ngắn gọn).
- Biên dịch để xuất mã nguồn hợp ngữ:

```
1 g++ -S -fverbose-asm source.cpp
```

- Mở file `source.s` và tìm đến nhãn `main`:

Input:

Không có dữ liệu đầu vào.

Output (Cấu trúc lệnh x86_64 tiêu biểu):

```
movl $5, -4(%rbp)           // Gán a = 5 vào bộ nhớ Stack
movl $10, -8(%rbp)          // Gán b = 10 vào bộ nhớ Stack
movl -4(%rbp), %edx          // Nạp a vào thanh ghi %edx
addl -8(%rbp), %eax          // Cộng b vào thanh ghi %eax
```

Câu hỏi thảo luận:

- Thanh ghi (Registers):** Tại sao CPU không thực hiện cộng trực tiếp hai ô nhớ mà phải đưa dữ liệu vào các thanh ghi như `%eax`, `%edx`?
- Instruction Set:** Lệnh `addl` tương ứng với toán tử nào trong C++? Nếu đổi sang kiến trúc ARM (điện thoại), lệnh này có còn là `addl` không?
- Tối ưu hóa mức thấp:** Thử biên dịch lại với `-O2 -S`. Tại sao các lệnh `movl` và `addl` cho các biến `a`, `b` lại biến mất?

Lưu ý:

Cờ `-fverbose-asm` sẽ giúp `g++` chèn thêm các chú thích tên biến C++ vào file Assembly, giúp dễ đối chiếu mã nguồn hơn.

III. Bài tập về nhà

Làm các bài tập chủ đề **Cú pháp & Ngữ nghĩa** (*[Syntax & Semantic]*) trên trang Coding. Liên hệ với giảng viên thực hành về thời hạn làm bài.

Nhắc lại: Để biên dịch và thực thi bài làm bằng C++, trong mã nguồn làm bài sẽ cần để ở dòng đầu tiên đoạn mã `// .cpp`, ví dụ như:

```
1 // .cpp
2 #include <iostream>
3 using namespace std;
4 ...
```